

Módulo 4 — Python para Preparação dos Dados

ROTEIRO DE AULA - Base PRF 2025 agrupada por ocorrência

Sessão 1 — Abertura, ambiente de trabalho e primeira leitura da base

Objetivo da sessão: Situar o módulo no CRISP-DM, montar o ambiente de trabalho e realizar a primeira leitura crítica da base bruta da PRF.

Bloco 0 — Abertura e contexto do módulo

Etapa 1 — Abertura do Módulo 4

O que aparece no slide

- Python para preparação dos dados
- Base PRF 2025 agrupada por ocorrência
- Entrega: notebook + duas bases + README

Etapa 2 — Onde o módulo se encaixa no CRISP-DM

O que aparece no slide

- Fase principal: Preparação dos Dados
- Também revisita a Compreensão dos Dados
- Prepara Modelagem, Avaliação e Comunicação

Etapa 3 — Produto oficial do módulo

O que aparece no slide

- Notebook executável
- Base analítica completa
- Base modelável sem data leakage
- README e decisões de tratamento

Bloco 1 — Preparação do ambiente de trabalho

Etapa 4 — Importar bibliotecas e configurar ambiente

O que aparece no slide

- pandas para tabelas
- numpy para regras numéricas
- matplotlib para checagens visuais
- Path para caminhos reproduzíveis

Comando para digitar/colar no notebook

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from pathlib import Path
from datetime import datetime
import unicodedata

pd.set_option("display.max_columns", 120)
pd.set_option("display.width", 160)
```

Atividade prática da turma

Criar a primeira célula do notebook com imports e opções de exibição.

Etapa 5 — Criar estrutura de pastas

O que aparece no slide

- Separar dados brutos e tratados
- Organizar notebooks, SQL e relatórios
- Evitar sobrescrever o CSV original

Comando para digitar/colar no notebook

```
RAIZ = Path(".")
PASTAS = [
    "dados_brutos", "dados_tratados", "notebooks", "sql",
    "dashboards", "relatorios", "apresentacao", "logs"
]
for pasta in PASTAS:
    (RAIZ / pasta).mkdir(parents=True, exist_ok=True)

print("Pastas verificadas/criadas:")
for pasta in PASTAS:
    print("-", RAIZ / pasta)
```

Atividade prática da turma

Executar a criação das pastas e conferir no explorador de arquivos.

Etapa 6 — Definir parâmetros do projeto

O que aparece no slide

- Centralizar nomes de arquivos
- Facilitar mudança de caminho
- Evitar strings espalhadas pelo notebook

Comando para digitar/colar no notebook

```
ARQUIVO_BRUTO = Path("dados_brutos/acidentes2025.csv")
ARQUIVO_BASE_ANALITICA = Path("dados_tratados/base_analitica_prf_2025.csv")
ARQUIVO_BASE_MODELADEL = Path("dados_tratados/base_modelavel_prf_2025.csv")
ARQUIVO_DICIONARIO = Path("dados_tratados/dicionario_variaveis_modulo4.csv")
ARQUIVO_DECISOES = Path("logs/decisoess_tratamento_modulo4.md")
ARQUIVO_README = Path("README.md")

SEPARADOR = ";"
ENCODING_ENTRADA = "latin1"
ENCODING_SAIDA = "utf-8-sig"
```

Atividade prática da turma

Criar a célula de parâmetros antes da leitura dos dados.

Bloco 2 — Leitura e primeira inspeção da base

Etapa 7 — Ler o CSV da PRF com fallback de encoding

O que aparece no slide

- CSV separado por ponto e vírgula
- Possível encoding latin1
- Leitura robusta evita travar a aula

Comando para digitar/colar no notebook

```
def ler_csv_prf(caminho, sep=";", encodings=("latin1", "utf-8", "utf-8-sig")):
    ultimo_erro = None
    for enc in encodings:
        try:
            print(f"Tentando leitura com encoding={enc}...")
```

```

        return pd.read_csv(caminho, sep=sep, encoding=enc, low_memory=False)
    except Exception as erro:
        ultimo_erro = erro
        print(f"Falhou com {enc}: {erro}")
        raise ultimo_erro

```

```

df = ler_csv_prf(ARQUIVO_BRUTO, sep=SEPARADOR)
df.head()

```

Atividade prática da turma

Ler o arquivo acidentes2025.csv a partir da pasta dados_brutos.

Etapa 8 — Padronizar nomes das colunas

O que aparece no slide

- Minúsculas
- Sem acentos
- Underline
- Compatibilização de grafias

Comando para digitar/colar no notebook

```

def normalizar_nome_coluna(nome):
    nome = str(nome).strip().lower()
    nome = unicodedata.normalize("NFKD", nome).encode("ascii", "ignore").decode("utf-8")
    nome = nome.replace(" ", "_").replace("-", "_").replace("/", "_")
    while "__" in nome:
        nome = nome.replace("__", "_")
    return nome.strip("_")

df.columns = [normalizar_nome_coluna(c) for c in df.columns]

# Compatibilização de grafias possíveis
renomear = {"condicao_meteorologica": "condicao_meteorologica"}
df = df.rename(columns={k: v for k, v in renomear.items() if k in df.columns})

print(df.columns.tolist())

```

Atividade prática da turma

Executar a padronização e registrar a lista final de colunas.

Etapa 9 — Conferir colunas esperadas

O que aparece no slide

- Validar estrutura da base
- Identificar campos faltantes
- Evitar erro silencioso nos próximos passos

Comando para digitar/colar no notebook

```

colunas_esperadas = [
    "data_inversa", "dia_semana", "horario", "uf", "br", "municipio",
    "causa_acidente", "tipo_acidente", "classificacao_acidente",
    "fase_dia", "condicao_meteorologica", "tipo_pista", "tracado_via",
    "uso_solo", "pessoas", "mortos", "feridos_leves",
    "feridos_graves", "feridos", "veiculos"
]

faltantes = [c for c in colunas_esperadas if c not in df.columns]
print("Colunas faltantes:", faltantes)

if faltantes:
    print("Atenção: ajuste nomes ou confirme o dicionário oficial da PRF usado no arquivo.")

```

Atividade prática da turma

Executar a conferência e comentar qualquer coluna faltante.

Etapa 10 — Retrato inicial da base

O que aparece no slide

- Dimensões
- Primeiras linhas
- Amostra aleatória
- Senso inicial de conteúdo

Comando para digitar/colar no notebook

```
print("Dimensões:", df.shape)
print("Linhas:", df.shape[0])
print("Colunas:", df.shape[1])

display(df.head())
display(df.sample(5, random_state=42))
```

Atividade prática da turma

Registrar no notebook a dimensão inicial da base.

Bloco 3 — Diagnóstico de qualidade dos dados

Etapa 11 — Tipos de dados e memória

O que aparece no slide

- Tipos inferidos pelo pandas
- Uso de memória
- Colunas numéricas como texto
- Datas ainda como objeto

Comando para digitar/colar no notebook

```
df.info(memory_usage="deep")

resumo_tipos = (
    df.dtypes.astype(str)
    .value_counts()
    .rename_axis("tipo")
    .reset_index(name="qtd_colunas")
)
display(resumo_tipos)
```

Atividade prática da turma

Executar df.info e listar problemas de tipo encontrados.

Etapa 12 — Diagnóstico de valores ausentes

O que aparece no slide

- Quantidade de nulos
- Percentual de nulos
- Priorizar campos relevantes
- Não confundir nulo com zero

Comando para digitar/colar no notebook

```
nulos = pd.DataFrame({
    "qtd_nulos": df.isna().sum(),
    "perc_nulos": df.isna().mean() * 100
})
```

```
}).sort_values("perc_nulos", ascending=False)

display(nulos[nulos["qtd_nulos"] > 0])
```

Atividade prática da turma

Gerar tabela de nulos e selecionar três colunas para discussão.

Etapa 13 — Diagnóstico e remoção de duplicidades

O que aparece no slide

- Duplicidade exata
- Remoção controlada
- Preservar rastreabilidade
- Registrar decisão

Comando para digitar/colar no notebook

```
qtd_duplicadas = df.duplicated().sum()
print("Duplicidades exatas:", qtd_duplicadas)

if qtd_duplicadas > 0:
    df = df.drop_duplicates().copy()
    print("Duplicidades removidas. Nova dimensão:", df.shape)
```

Atividade prática da turma

Verificar duplicidades e registrar a decisão no notebook.

Etapa 14 — Cardinalidade das categorias

O que aparece no slide

- Quantas categorias por campo
- Município tende a ser alto
- Causa e tipo exigem cuidado
- Categorias raras podem distorcer análise

Comando para digitar/colar no notebook

```
categoricas = df.select_dtypes(include="object").columns
cardinalidade = (
    df[categoricas]
    .nunique(dropna=True)
    .sort_values(ascending=False)
    .reset_index()
)
cardinalidade.columns = ["variavel", "qtd_categorias"]
display(cardinalidade.head(30))
```

Atividade prática da turma

Gerar cardinalidade e destacar variáveis com muitas categorias.

Sessão 2 — Transformações e variáveis derivadas

Objetivo da sessão: Converter tipos, tratar datas e horários, padronizar textos e tratar nulos para deixar a base pronta para receber o alvo.

Bloco 4 — Transformações e criação de variáveis derivadas

Etapa 15 — Converter colunas numéricas

O que aparece no slide

- Contagens de vítimas
- BR e km
- Conversão segura
- Nulos podem surgir por erro de formato

Comando para digitar/colar no notebook

```
colunas_numericas = [
    "br", "km", "pessoas", "mortos", "feridos", "feridos_leves",
    "feridos_graves", "ilesos", "ignorados", "veiculos"
]

for coluna in colunas_numericas:
    if coluna in df.columns:
        df[coluna] = pd.to_numeric(df[coluna], errors="coerce")

print(df[[c for c in colunas_numericas if c in df.columns]].dtypes)
```

Atividade prática da turma

Converter colunas numéricas essenciais e conferir tipos.

Etapa 16 — Tratar datas e criar variáveis temporais

O que aparece no slide

- data_inversa para datetime
- ano, mês e trimestre
- dia da semana numérico
- fim de semana

Comando para digitar/colar no notebook

```
df["data_inversa"] = pd.to_datetime(df["data_inversa"], errors="coerce")

df["ano"] = df["data_inversa"].dt.year
df["mes"] = df["data_inversa"].dt.month
df["trimestre"] = df["data_inversa"].dt.quarter
df["dia_semana_num"] = df["data_inversa"].dt.dayofweek
df["fim_de_semana"] = df["dia_semana_num"].isin([5, 6]).astype(int)

display(df[["data_inversa", "ano", "mes", "trimestre", "dia_semana_num",
"fim_de_semana"]].head())
```

Atividade prática da turma

Criar as variáveis temporais e conferir uma amostra.

Etapa 17 — Tratar horário e criar turno

O que aparece no slide

- Extrair hora
- Classificar madrugada, manhã, tarde e noite

- Tratar horários inválidos
- Gerar variável interpretável

Comando para digitar/colar no notebook

```
horario_limpo = df["horario"].astype(str).str.strip()
df["hora"] = pd.to_datetime(horario_limpo, format="%H:%M:%S", errors="coerce").dt.hour

# Tentativa alternativa para arquivos com HH:MM
faltou_hora = df["hora"].isna()
if faltou_hora.any():
    df.loc[faltou_hora, "hora"] = pd.to_datetime(
        horario_limpo[faltou_hora], format="%H:%M", errors="coerce"
    ).dt.hour

def classificar_turno(hora):
    if pd.isna(hora):
        return "IGNORADO"
    if 0 <= hora <= 5:
        return "MADRUGADA"
    if 6 <= hora <= 11:
        return "MANHA"
    if 12 <= hora <= 17:
        return "TARDE"
    return "NOITE"

df["turno"] = df["hora"].apply(classificar_turno)
display(df[["horario", "hora", "turno"]].head())
```

Atividade prática da turma

Criar a variável turno e revisar frequências.

Etapa 18 — Criar faixa horária

O que aparece no slide

- Blocos de 3 horas
- Reduz granularidade
- Ajuda em padrões temporais
- Útil para modelo explicável

Comando para digitar/colar no notebook

```
def criar_faixa_horaria(hora):
    if pd.isna(hora):
        return "IGNORADO"
    inicio = int(hora // 3) * 3
    fim = inicio + 2
    return f"{inicio:02d}h-{fim:02d}h"

df["faixa_horaria"] = df["hora"].apply(criar_faixa_horaria)

display(df["faixa_horaria"].value_counts(dropna=False).sort_index())
```

Atividade prática da turma

Criar faixa_horaria e conferir distribuição.

Etapa 19 — Padronizar textos

O que aparece no slide

- Remover espaços
- Maiúsculas
- Vazios como nulos
- Categorias mais consistentes

Comando para digitar/colar no notebook

```
colunas_texto = df.select_dtypes(include="object").columns

for coluna in colunas_texto:
    df[coluna] = (
        df[coluna]
        .astype("string")
        .str.strip()
        .str.upper()
    )
    df[coluna] = df[coluna].replace({"": pd.NA, "NaN": pd.NA, "NONE": pd.NA, "NULL": pd.NA})

print("Colunas textuais padronizadas:", len(colunas_texto))
```

Atividade prática da turma

Padronizar colunas textuais e comparar value_counts antes/depois em uma coluna.

Etapa 20 — Tratar nulos categóricos

O que aparece no slide

- Usar IGNORADO em campos explicativos
- Evitar perda de linhas
- Preservar informação de ausência
- Documentar decisão

Comando para digitar/colar no notebook

```
categoricas_importantes = [
    "uf", "municipio", "causa_acidente", "tipo_acidente", "fase_dia",
    "condicao_meteorologica", "tipo_pista", "tracado_via", "uso_solo",
    "classificacao_acidente", "dia_semana"
]

for coluna in categoricas_importantes:
    if coluna in df.columns:
        df[coluna] = df[coluna].fillna("IGNORADO")

print(df[categoricas_importantes].isna().sum().sort_values(ascending=False))
```

Atividade prática da turma

Preencher categorias importantes e conferir nulos remanescentes.

Etapa 21 — Tratar nulos numéricos de contagem

O que aparece no slide

- Contagens de vítimas
- Zero como hipótese operacional
- Atenção à semântica da coluna
- Registrar decisão

Comando para digitar/colar no notebook

```
contagens_vitimas = ["mortos", "feridos", "feridos_leves", "feridos_graves", "pessoas",
                     "veiculos"]

for coluna in contagens_vitimas:
    if coluna in df.columns:
        df[coluna] = df[coluna].fillna(0)

print(df[[c for c in contagens_vitimas if c in df.columns]].isna().sum())
```

Atividade prática da turma

Preencher nulos em contagens e registrar a regra adotada.

Sessão 3 — Variável-alvo e indicadores de gravidade

Objetivo da sessão: Construir e validar a variável-alvo `acidente_fatal`, criar indicadores derivados e realizar checagens de consistência.

Bloco 5 — Variável-alvo e indicadores de gravidade

Etapa 22 — Criar variável-alvo `acidente_fatal`

O que aparece no slide

- 1 quando mortos ≥ 1
- 0 quando mortos = 0
- Centro do problema binário
- Base para análises posteriores

Comando para digitar/colar no notebook

```
df["acidente_fatal"] = np.where(df["mortos"] >= 1, 1, 0)

validacao_alvo =
df["acidente_fatal"].value_counts(dropna=False).rename_axis("acidente_fatal").reset_index(name="qtd")
validacao_alvo["perc"] = validacao_alvo["qtd"] / validacao_alvo["qtd"].sum() * 100
display(validacao_alvo)
```

Atividade prática da turma

Criar `acidente_fatal` e conferir distribuição absoluta e percentual.

Etapa 23 — Validar logicamente o alvo

O que aparece no slide

- Testar regra
- Detectar inversões
- Detectar nulos indevidos
- Usar `assert`

Comando para digitar/colar no notebook

```
violacoes = df.loc[
    ((df["mortos"] >= 1) & (df["acidente_fatal"] != 1)) |
    ((df["mortos"] == 0) & (df["acidente_fatal"] != 0))
]

print("Violações da regra do alvo:", len(violacoes))
assert len(violacoes) == 0, "Há erro na criação de acidente_fatal."
```

Atividade prática da turma

Executar o teste lógico e registrar o resultado.

Etapa 24 — Criar indicadores de gravidade

O que aparece no slide

- `total_vitimas`
- `acidente_grave`
- `indice_gravidade`
- Uso: análise e Power BI

Comando para digitar/colar no notebook

```
df["total_vitimas"] = df["mortos"] + df["feridos_leves"] + df["feridos_graves"]
```

```
df["acidente_grave"] = np.where(
    (df["mortos"] >= 1) | (df["feridos_graves"] >= 1),
    1,
    0
)

df["indice_gravidade"] = (
    df["mortos"] * 3
    + df["feridos_graves"] * 2
    + df["feridos_leves"]
)

display(df[["mortos", "feridos_leves", "feridos_graves", "total_vitimas",
"indice_gravidade"]].head())
```

Atividade prática da turma

Criar indicadores de gravidade e conferir amostra.

Etapa 25 — Criar BR formatada e chave de localidade

O que aparece no slide

- BR-000
- BR ignorada
- UF + município + BR
- Facilita filtros e agrupamentos

Comando para digitar/colar no notebook

```
def formatar_br(valor):
    if pd.isna(valor) or valor == 0:
        return "BR-IGNORADA"
    return f"BR-{int(valor):03d}"

df["br_formatada"] = df["br"].apply(formatar_br)

df["chave_localidade"] = (
    df["uf"].astype(str) + "_" +
    df["municipio"].astype(str) + "_" +
    df["br_formatada"].astype(str)
)

display(df[["uf", "municipio", "br", "br_formatada", "chave_localidade"]].head())
```

Atividade prática da turma

Criar br_formatada e chave_localidade.

Etapa 26 — Checagens rápidas após transformação

O que aparece no slide

- Linhas e colunas
- Quantidade de fatais
- Taxa de fatalidade
- Totais de mortos e feridos

Comando para digitar/colar no notebook

```
checagens = {
    "linhas": len(df),
    "colunas": df.shape[1],
    "acidentes_fatais": int(df["acidente_fatal"].sum()),
    "taxa_fatalidade": float(df["acidente_fatal"].mean()),
    "total_mortos": int(df["mortos"].sum()),
```

```
"total_feridos": int(df["feridos"].sum()) if "feridos" in df.columns else None,
}
checagens
```

Atividade prática da turma

Executar checagens e comparar com expectativa inicial.

Etapa 27 — Ranking rápido de categorias

O que aparece no slide

- Causas mais frequentes
- Tipos mais frequentes
- Conferência de categorias
- Preparação para EDA

Comando para digitar/colar no notebook

```
def ranking_categoria(base, coluna, n=10):
    return (
        base[coluna]
        .value_counts(dropna=False)
        .head(n)
        .rename_axis(coluna)
        .reset_index(name="qtd")
    )

display(ranking_categoria(df, "causa_acidente", 10))
display(ranking_categoria(df, "tipo_acidente", 10))
```

Atividade prática da turma

Gerar ranking de causa_acidente e tipo_acidente.

Etapa 28 — Taxa fatal por categoria

O que aparece no slide

- Proporção, não apenas frequência
- Filtro por mínimo de registros
- Evitar subgrupos minúsculos
- Preparação para análise bivariada

Comando para digitar/colar no notebook

```
def taxa_fatal_por_categoria(base, coluna, min_registros=30):
    tab = base.groupby(coluna).agg(
        qtd_acidentes=("acidente_fatal", "size"),
        qtd_fatais=("acidente_fatal", "sum"),
        taxa_fatal=("acidente_fatal", "mean")
    ).reset_index()
    tab = tab[tab["qtd_acidentes"] >= min_registros]
    return tab.sort_values("taxa_fatal", ascending=False)

display(taxa_fatal_por_categoria(df, "tipo_acidente", min_registros=30).head(10))
```

Atividade prática da turma

Calcular taxa fatal por tipo_acidente com mínimo de 30 registros.

Etapa 29 — Gráfico simples de conferência

O que aparece no slide

- Distribuição do alvo
- Visualização rápida
- Não substitui EDA

- Ajuda a detectar desequilíbrio

Comando para digitar/colar no notebook

```
ax = df["acidente_fatal"].value_counts().sort_index().plot(kind="bar")
ax.set_title("Distribuição da variável-alvo acidente_fatal")
ax.set_xlabel("acidente_fatal")
ax.set_ylabel("Quantidade de acidentes")
plt.show()
```

Atividade prática da turma

Gerar gráfico de acidente_fatal.

Sessão 4 — Construção das bases finais e documentação

Objetivo da sessão: Separar a base analítica da base modelável, eliminar data leakage, exportar os arquivos finais e documentar as decisões do projeto.

Bloco 6 — Construção das duas bases (analítica e modelável)

Etapa 30 — Conceito de base analítica completa

O que aparece no slide

- Usada para EDA e Power BI
- Pode conter mortos e feridos
- Pode conter indicadores derivados
- Foco em descrição e comunicação

Atividade prática da turma

Discussão: quais variáveis são úteis para dashboard?

Etapa 31 — Gerar base analítica completa

O que aparece no slide

- Cópia da base tratada
- Preserva variáveis descritivas
- Inclui indicadores de gravidade
- Será usada nos módulos 5 e 6

Comando para digitar/colar no notebook

```
base_analitica = df.copy()

print("Base analítica:", base_analitica.shape)
print("Colunas:", base_analitica.columns.tolist())
```

Atividade prática da turma

Criar base_analitica e verificar dimensões.

Etapa 32 — Conceito de base modelável

O que aparece no slide

- Usada na árvore de decisão
- Apenas variáveis explicativas
- Inclui acidente_fatal como alvo
- Exclui variáveis derivadas do desfecho

Atividade prática da turma

Classificar variáveis em permitidas, proibidas e duvidosas.

Etapa 33 — Selecionar variáveis modeláveis

O que aparece no slide

- UF, BR, município
- Tempo e turno
- Causa, tipo, pista, clima e traçado
- Alvo acidente_fatal

Comando para digitar/colar no notebook

```
variaveis_modelaveis = [
    "uf", "br_formatada", "municipio", "mes", "trimestre",
    "dia_semana", "dia_semana_num", "fim_de_semana",
    "hora", "faixa_horaria", "turno", "fase_dia",
    "causa_acidente", "tipo_acidente", "condicao_metereologica",
    "tipo_pista", "tracado_via", "uso_solo",
    "acidente_fatal"
]

variaveis_modelaveis = [c for c in variaveis_modelaveis if c in df.columns]
base_modelavel = df[variaveis_modelaveis].copy()

print("Base modelável:", base_modelavel.shape)
```

Atividade prática da turma

Criar base_modelavel com variáveis permitidas.

Etapa 34 — Verificar data leakage

O que aparece no slide

- mortos fora da modelável
- feridos fora da modelável
- total_vitimas fora da modelável
- indice_gravidade fora da modelável

Comando para digitar/colar no notebook

```
variaveis_proibidas = [
    "mortos", "feridos", "feridos_leves", "feridos_graves",
    "total_vitimas", "indice_gravidade", "acidente_grave",
    "classificacao_acidente"
]

def verificar_data_leakage(base, proibidas):
    presentes = [c for c in proibidas if c in base.columns]
    if presentes:
        raise ValueError(f"Data leakage detectado: {presentes}")
    return "OK – nenhuma variável proibida encontrada."

verificar_data_leakage(base_modelavel, variaveis_proibidas)
```

Atividade prática da turma

Executar verificação de leakage e registrar OK.

Etapa 35 — Tratamento final de nulos na base modelável

O que aparece no slide

- Categóricas: IGNORADO
- Numéricas: -1 quando necessário
- Alvo preservado
- Evitar erro na modelagem

Comando para digitar/colar no notebook

```
for coluna in base_modelavel.columns:
    if coluna == "acidente_fatal":
        continue
    if base_modelavel[coluna].dtype == "object" or str(base_modelavel[coluna].dtype) == "string":
        base_modelavel[coluna] = base_modelavel[coluna].fillna("IGNORADO")
    else:
        base_modelavel[coluna] = base_modelavel[coluna].fillna(-1)
```

```
print(base_modelavel.isna().sum().sort_values(ascending=False).head())
```

Atividade prática da turma

Tratar nulos remanescentes na base_modelavel.

Etapa 36 — Exportar bases tratadas

O que aparece no slide

- CSV em dados_tratados
- Separador ;
- utf-8-sig
- Não sobrescrever dados brutos

Comando para digitar/colar no notebook

```
base_analitica.to_csv(ARQUIVO_BASE_ANALITICA, index=False, sep=SEPARADOR,
encoding=ENCODING_SAIDA)
base_modelavel.to_csv(ARQUIVO_BASE_MODELAVEL, index=False, sep=SEPARADOR,
encoding=ENCODING_SAIDA)

print("Arquivos exportados:")
print("-", ARQUIVO_BASE_ANALITICA)
print("-", ARQUIVO_BASE_MODELAVEL)
```

Atividade prática da turma

Exportar base analítica e base modelável.

Etapa 37 — Reabrir arquivos exportados

O que aparece no slide

- Validação pós-exportação
- Conferir dimensões
- Garantir legibilidade
- Detectar erro de encoding

Comando para digitar/colar no notebook

```
valid_analitica = pd.read_csv(ARQUIVO_BASE_ANALITICA, sep=SEPARADOR, encoding=ENCODING_SAIDA)
valid_modelavel = pd.read_csv(ARQUIVO_BASE_MODELAVEL, sep=SEPARADOR, encoding=ENCODING_SAIDA)

print("Analítica reaberta:", valid_analitica.shape)
print("Modelável reaberta:", valid_modelavel.shape)
assert len(valid_analitica) == len(base_analitica)
assert len(valid_modelavel) == len(base_modelavel)
```

Atividade prática da turma

Reabrir as bases e validar dimensões.

Bloco 7 — Documentação do projeto

Etapa 38 — Gerar dicionário das variáveis criadas

O que aparece no slide

- Descrição das novas variáveis
- Uso previsto
- Apoio à avaliação
- Apoio aos módulos seguintes

Comando para digitar/colar no notebook


```
linhas_dic = [
    {"variavel": "acidente_fatal", "descricao": "1 se mortos >= 1; 0 se mortos = 0", "uso":
    "alvo"},
    {"variavel": "total_vitimas", "descricao": "mortos + feridos leves + feridos graves", "uso":
    "analise/dashboard"},
    {"variavel": "indice_gravidade", "descricao": "mortos*3 + feridos_graves*2 + feridos_leves",
    "uso": "analise/dashboard"},
    {"variavel": "br_formatada", "descricao": "BR padronizada no formato BR-000", "uso":
    "analise/modelagem"},
    {"variavel": "chave_localidade", "descricao": "UF + município + BR formatada", "uso":
    "analise/dashboard"},
]

dicionario = pd.DataFrame(linhas_dic)
dicionario.to_csv(ARQUIVO_DICIONARIO, index=False, sep=SEPARADOR, encoding=ENCODING_SAIDA)
display(dicionario)
```

Atividade prática da turma

Gerar dicionário_variaveis_modulo4.csv.

Etapa 39 — Registrar decisões de tratamento

O que aparece no slide

- Data da geração
- Regras adotadas
- Arquivos gerados
- Justificativa metodológica

Comando para digitar/colar no notebook

```
texto_decisoes = f"""
# Decisões de tratamento – Módulo 4

Data de geração: {datetime.now().strftime('%Y-%m-%d %H:%M')}

## Principais decisões
- Nomes de colunas padronizados para minúsculas, sem acentos e com underline.
- Colunas numéricas convertidas com `pd.to_numeric(errors='coerce')`.
- Datas convertidas com `pd.to_datetime(errors='coerce')`.
- Categorias ausentes relevantes preenchidas como IGNORADO.
- Variável-alvo: acidente_fatal = 1 quando mortos >= 1.
- Base modelável exclui variáveis derivadas do desfecho.

## Arquivos gerados
- {ARQUIVO_BASE_ANALITICA}
- {ARQUIVO_BASE_MODELAVEL}
- {ARQUIVO_DICIONARIO}
"""

ARQUIVO_DECISOES.write_text(texto_decisoes, encoding="utf-8")
print(ARQUIVO_DECISOES)
```

Atividade prática da turma

Criar arquivo de decisões em logs.

Etapa 40 — Criar README do projeto

O que aparece no slide

- Objetivo
- Variável-alvo
- Bases geradas
- Observação sobre data leakage

Comando para digitar/colar no notebook

```
readme = f"""
# Projeto PRF 2025 – Preparação dos Dados

## Objetivo
Preparar os dados de acidentes da PRF 2025 para análise exploratória, Power BI e árvore de
decisão explicável.

## Variável-alvo
`acidente_fatal = 1` quando `mortos >= 1`; caso contrário, `acidente_fatal = 0`.

## Bases geradas
- `{ARQUIVO_BASE_ANALITICA}`: base completa para EDA e Power BI.
- `{ARQUIVO_BASE_MODELAVEL}`: base para modelagem, sem data leakage.

## Observação metodológica
A base modelável exclui mortos, feridos, total_vitimas, indice_gravidade e variáveis diretamente
derivadas do desfecho.
"""

ARQUIVO_README.write_text(readme, encoding="utf-8")
print("README criado:", ARQUIVO_README)
```

Atividade prática da turma

Criar README.md e revisar conteúdo.

Etapa 41 — Resumo final da preparação

O que aparece no slide

- Dimensões das bases
- Taxa global de fatalidade
- Conferência final
- Evidência para avaliação

Comando para digitar/colar no notebook

```
resumo_final = pd.DataFrame([
    {"item": "linhas_base_analitica", "valor": len(base_analitica)},
    {"item": "colunas_base_analitica", "valor": base_analitica.shape[1]},
    {"item": "linhas_base_modelavel", "valor": len(base_modelavel)},
    {"item": "colunas_base_modelavel", "valor": base_modelavel.shape[1]},
    {"item": "taxa_global_acidente_fatal", "valor": base_modelavel["acidente_fatal"].mean()},
])

display(resumo_final)
```

Atividade prática da turma

Gerar o resumo final no notebook.

Sessão 5 — Boas práticas, erros comuns e encerramento

Objetivo da sessão: Consolidar boas práticas de notebook, revisar erros comuns, aplicar o checklist final e conectar o módulo com o restante do curso.

Bloco 8 — Boas práticas e erros comuns

Etapa 47 — Boas práticas em notebooks

O que aparece no slide

- Executar do início ao fim
- Células em ordem lógica
- Comentários curtos
- Sem caminhos pessoais fixos

Atividade prática da turma

Reiniciar kernel e executar tudo para validar reprodutibilidade.

Etapa 48 — Erros comuns na leitura do CSV

O que aparece no slide

- Separador errado
- Encoding errado
- Colunas em uma única coluna
- Acentuação quebrada

Atividade prática da turma

Revisar se `df.shape` e `df.head` fazem sentido.

Etapa 49 — Erros comuns no alvo

O que aparece no slide

- Inverter 0 e 1
- Usar feridos no lugar de mortos
- Não tratar nulos
- Não conferir distribuição

Atividade prática da turma

Explicar verbalmente a regra de `acidente_fatal`.

Etapa 50 — Erros comuns de data leakage

O que aparece no slide

- Manter mortos na modelável
- Manter `total_vitimas`
- Manter `indice_gravidade`
- Usar classificação do acidente sem crítica

Atividade prática da turma

Marcar variáveis proibidas no dicionário do projeto.

Bloco 9 — Checklist, critérios de qualidade e conexões com os próximos módulos

Etapa 51 — Checklist final da entrega

O que aparece no slide

- Notebook executa sem erro
- Bases em dados_tratados
- README criado
- Decisões registradas
- Leakage ausente

Atividade prática da turma

Conferir todos os itens antes da entrega.

Etapa 52 — Critérios de qualidade

O que aparece no slide

- Correção técnica
- Reprodutibilidade
- Documentação
- Separação correta das bases
- Prevenção de vazamento

Atividade prática da turma

Autoavaliar a entrega usando os critérios.

Etapa 53 — Conexão com Módulo 5

O que aparece no slide

- Base analítica alimenta EDA
- Indicadores serão explorados
- Cruzamentos com acidente_fatal
- Hipóteses serão registradas

Atividade prática da turma

Listar três perguntas que a base analítica permite responder.

Etapa 54 — Conexão com Módulo 6

O que aparece no slide

- Power BI usa base analítica
- Medidas dependem de campos tratados
- Filtros usam variáveis padronizadas
- Dashboard depende da qualidade da base

Atividade prática da turma

Conferir se a base analítica contém campos para filtros de UF, BR, município e mês.

Etapa 55 — Conexão com Módulo 7

O que aparece no slide

- Base modelável alimenta árvore
- Sem data leakage
- Foco em explicabilidade

- Lift e cobertura virão depois

Atividade prática da turma

Conferir se a base modelável contém apenas variáveis explicativas e o alvo.

Etapa 56 — Fechamento do Módulo 4

O que aparece no slide

- Dados preparados com rigor
- Duas bases com finalidades distintas
- Notebook reprodutível
- Documentação pronta para avaliação

Atividade prática da turma

Submeter pacote final do módulo.